

Comparative Analysis of Network Intrusion Detection Systems for Small Business Environments

Abdulrahman Rahaq

ABSTRACT

Small businesses face the same network attacks as large organisations, but rarely have the budget, staff, or hardware to defend themselves the same way. This project compares three widely-used open-source Intrusion Detection Systems — Snort, Suricata, and Zeek — on a single 2 GB virtual machine built to resemble a real SMB setup.

Each tool was tested against four common attacks with no expert tuning: a port scan, an ICMP flood, a SYN flood, and an SSH brute force. Snort detected three of the four. Suricata crashed on its default rule set and only ran after manual reduction. Zeek produced no real-time alerts but captured every SSH attempt in its log files. Running two tools simultaneously during a flood pushed the host to 83% RAM and took its SSH service offline.

INTRODUCTION

Almost every organisation runs a network connection of some kind, and the volume of attacks targeting those networks has grown steadily for over a decade. An Intrusion Detection System (IDS) watches network traffic and flags anything that looks malicious or out of place. For larger organisations it is one layer of a much larger defensive stack. For small and medium-sized businesses (SMBs), it may be the only network monitoring tool they ever deploy.

Three open-source IDS tools dominate both academic literature and real-world deployments. Snort was first described by Roesch (1999) as a lightweight signature-based engine — it compares network packets against a set of known-bad patterns and raises an alert when one matches. Suricata, developed roughly a decade later, extends this approach with multi-threaded processing so it can keep pace with faster networks. Zeek takes a fundamentally different approach: rather than matching rules and firing alerts, it decodes each protocol conversation and writes structured log files for later analysis.

Research Questions

- RQ1

Which tool is most reliable on 2 GB of RAM with no expert tuning?
- RQ2

What configuration barriers does each tool present to a generalist administrator?
- RQ3

Can two tools run simultaneously during an active attack without breaking something?
- RQ4

What are the trade-offs between signature-based alerting and Zeek's log-based approach?

METHODOLOGY

Quantitative experimental design in a controlled VirtualBox lab. Three VMs share one isolated NAT network. Each IDS tool was tested independently against four identical attacks on the same 2 GB monitoring host, then a concurrent test ran two tools simultaneously during a live attack.

Ethics

All experiments ran inside a self-contained virtual lab. No external systems, live data, or real credentials were involved..

REFERENCES

Roesch (1999) USENIX LISA · Paxson (1999) Comput. Netw. · Sommer & Paxson (2010) IEEE S&P · Garcia-Teodoro et al. (2009) Comput. Secur. · Hindy et al. (2018) arXiv:1806.03517 · Lukaseder et al. (2018) ACM SAC · Shah & Issac (2017) FGCS · Waleed et al. (2022) Comput. Netw. · Ferriyan et al. (2021) Appl. Sci. · Ghazi et al. (2024) IJICT · Al-Dulaimi et al. (2023) IET · Ahmed et al. (2023) JNCA

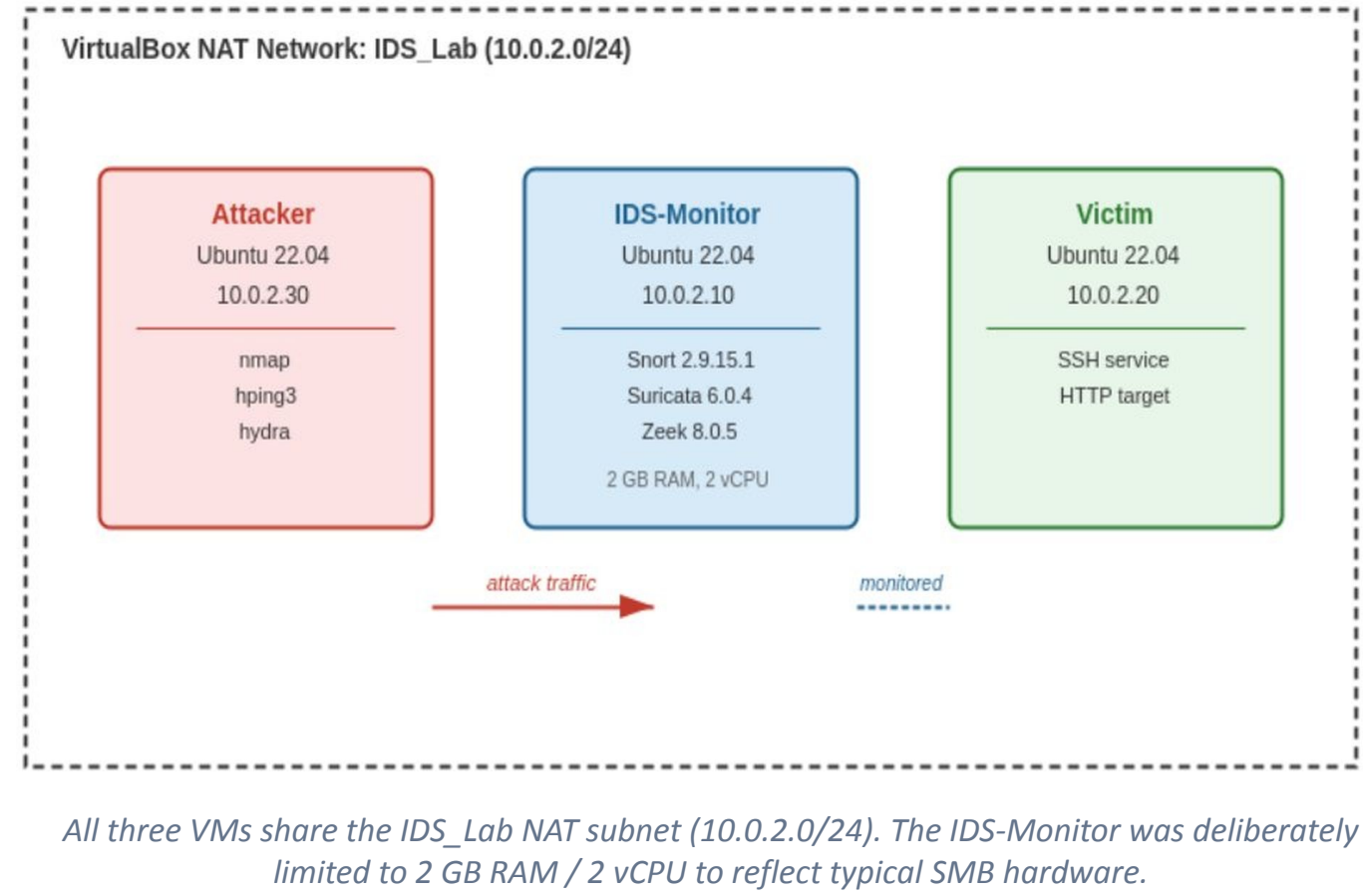
MATERIALS & METHODS

Quantitative experimental design in a controlled virtual lab. All three IDS tools were installed on the same monitoring host and tested against four identical attacks — no tool received any advantage. A concurrent test then ran two tools at the same time to assess multi-tool feasibility on SMB-grade hardware.

Test Environment

Machine	OS	RAM	CPU	IP
IDS-Monitor	Ubuntu 22.04	2 GB	2 vCPU	10.0.2.10
Victim	Ubuntu 22.04	1 GB	1 vCPU	10.0.2.20
Attacker	Ubuntu 22.04	1 GB	1 vCPU	10.0.2.30

Figure 1. Virtual Network Topology



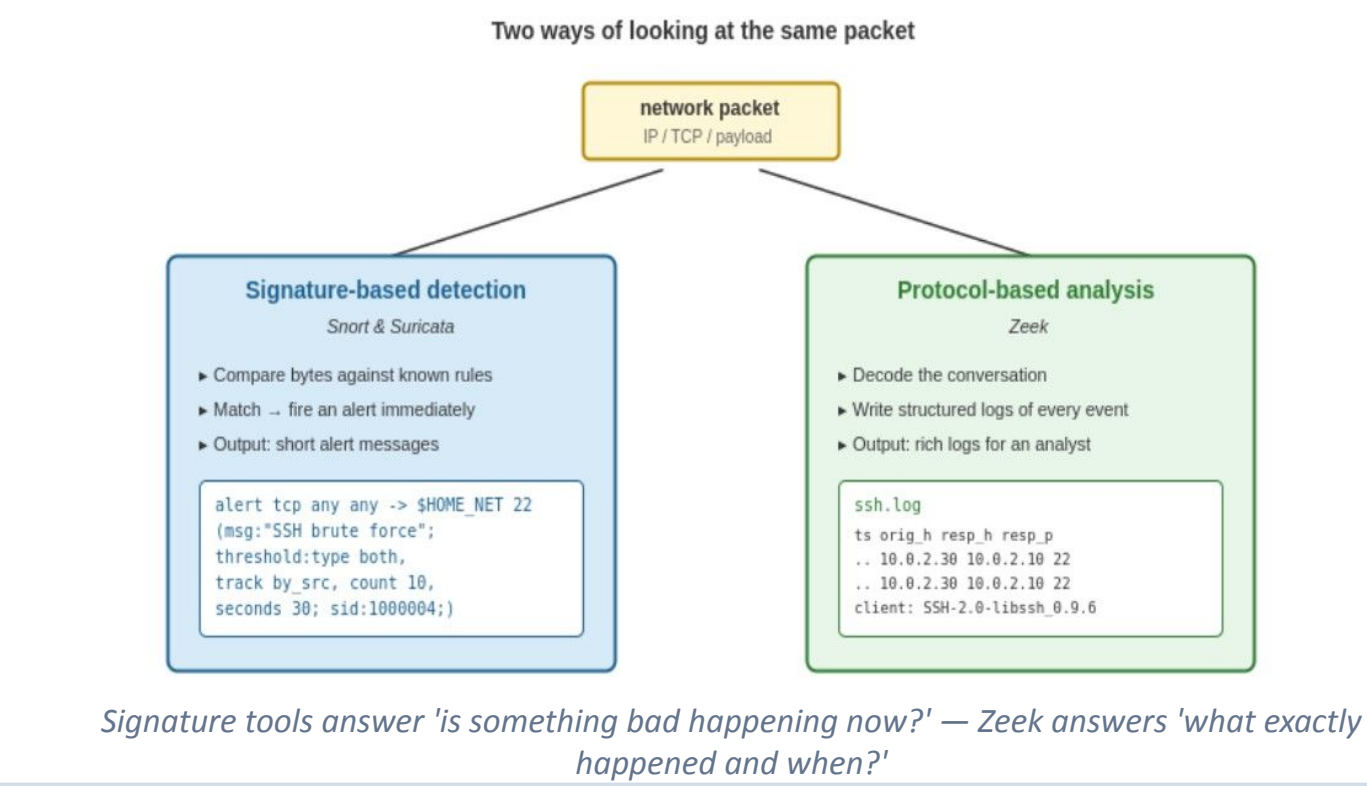
Tools Installed on IDS-Monitor

Tool	Version	Source
Snort	2.9.15.1	Ubuntu 22.04 universe repository
Suricata	6.0.4	Ubuntu 22.04 universe + suricata-update (ET Open rules)
Zeek	8.0.5	openSUSE Build Service repository

Attacks Used

Attack	Command	Duration
Port scan	<code>nmap -sS 10.0.2.10</code>	~30 s
ICMP flood	<code>hping3 -1 --flood 10.0.2.10</code>	30 s
SYN flood	<code>hping3 -S --flood -p 80 10.0.2.10</code>	30 s
SSH brute force	<code>hydra -l idsuser -P rockyou.txt ssh://</code>	2–3 min

Figure 2. Signature-Based vs. Protocol-Based Detection



RESULTS

Figure 3. Detection / Logging Outcome by Tool and Attack Type

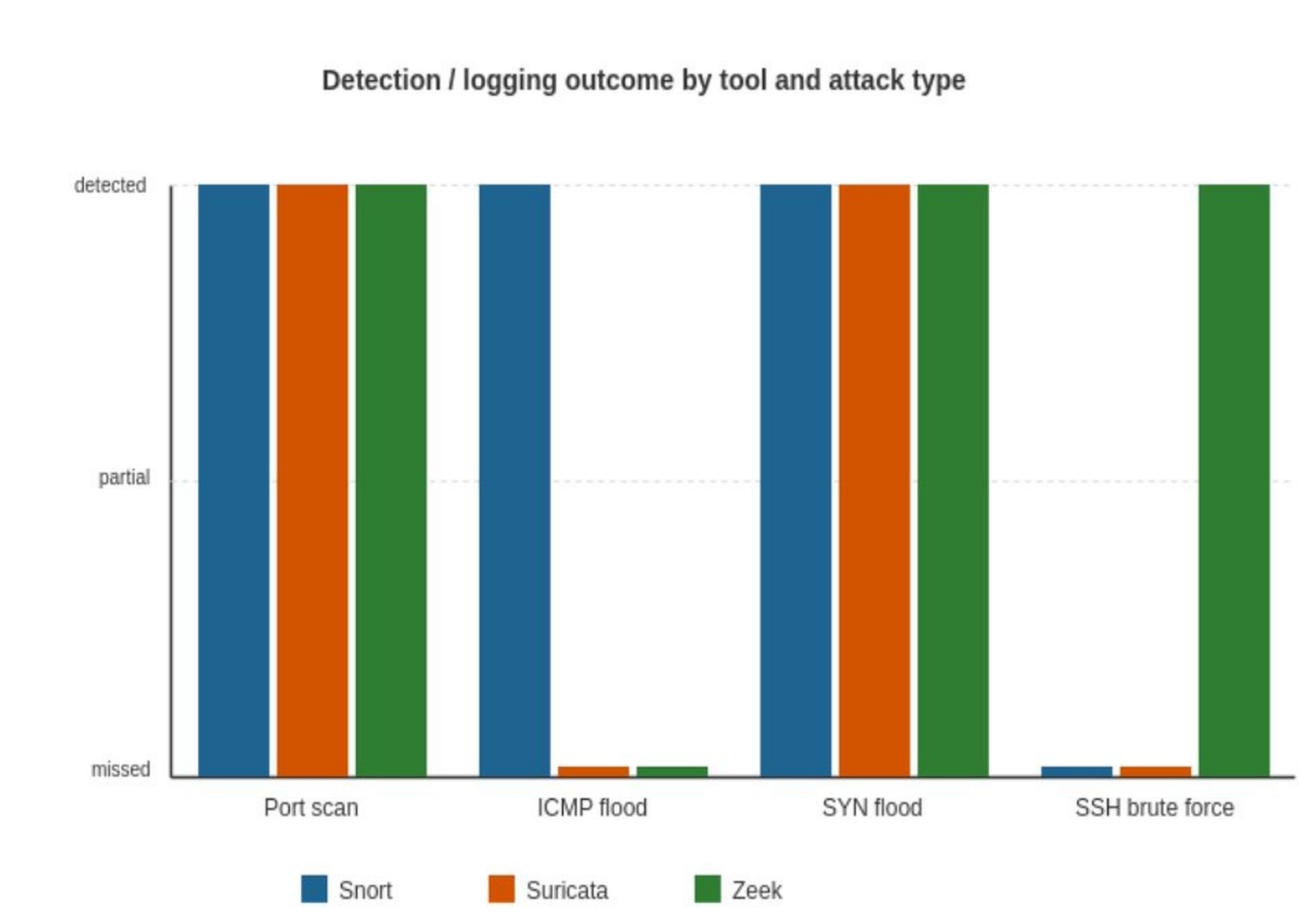
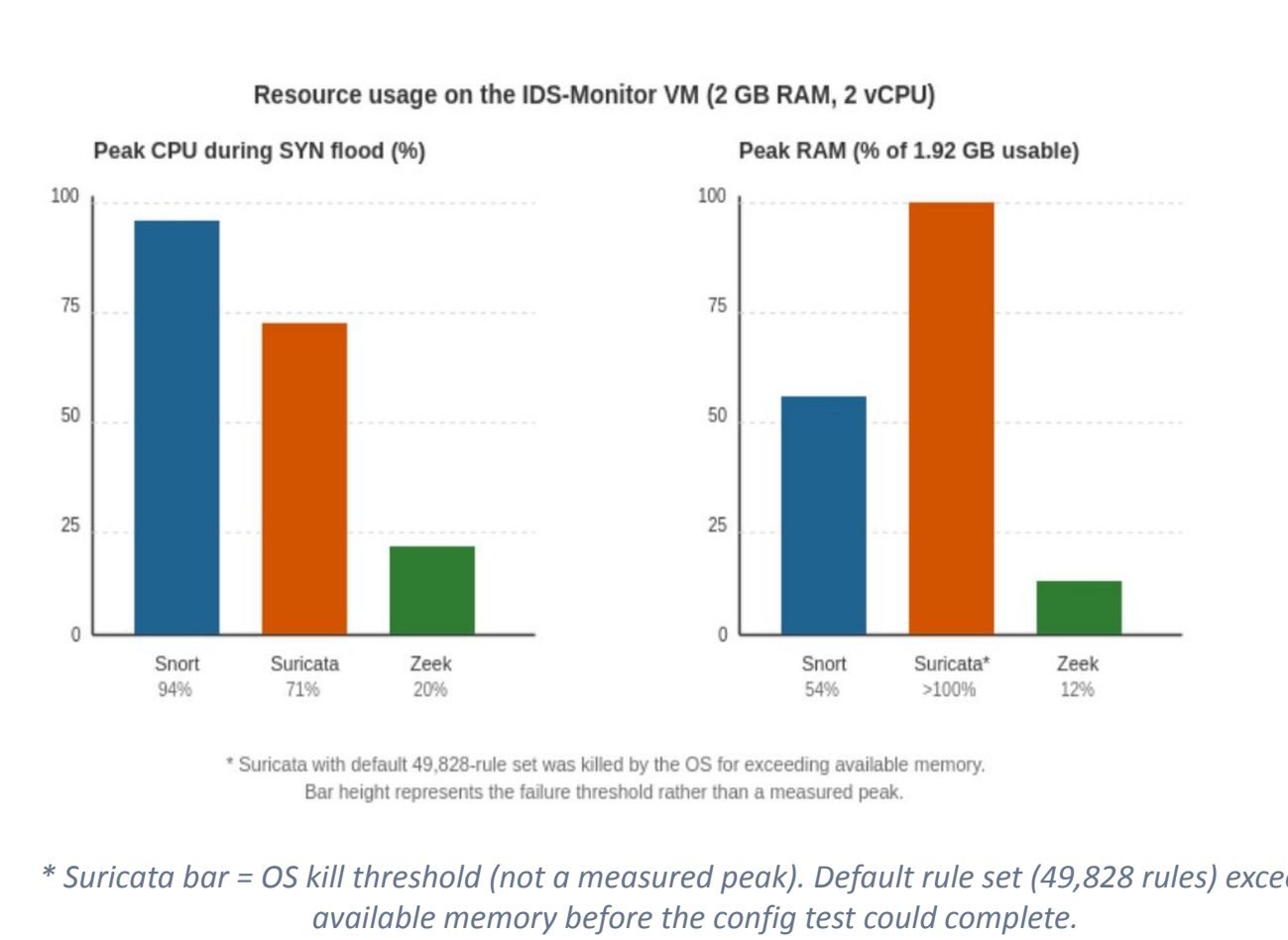


Figure 4. Peak Resource Usage on the IDS-Monitor VM (2 GB RAM, 2 vCPU)



Experiment 1: Snort

3/4 Detected

- Ran on default rules — only HOME_NET required in snort.conf
- Detected port scan, ICMP flood and SYN flood immediately
- Peak CPU 94% during floods; RAM peaked at ~1.04 GB
- SSH brute force: zero alerts — no SSH-specific logic in default rule set

Experiment 2: Suricata

2/4 Detected

- Default 49,828-rule set killed by the OS on startup — output: 'Killed'
- Ran only after manual reduction to 4 targeted custom rules
- Detected port scan and SYN flood; ICMP rule silently failed to fire
- SSH brute force not detected — needs application-layer rule parsing

Experiment 3: Zeek

3/4 Detected

- No real-time alerts — produces structured log files only
- ssh.log captured every brute-force attempt including the Hydra client banner
- conn.log recorded port scan and SYN flood via REJ connection states
- ICMP flood not captured clearly under default config at 2 GB RAM

CONCLUSION

Attack	Snort	Suricata	Zeek
Port scan	✓ Detected	✓ Detected	✓ Logged
ICMP flood	✓ Detected	✗ Missed	✗ Missed
SYN flood	✓ Detected	✓ Detected	✓ Logged
SSH brute force	✗ Missed	✗ Missed	✓ Logged

No single tool wins everywhere — the disagreements align with each tool's underlying detection paradigm. Snort ran reliably out of the box and gave the fastest alerts — but it has no SSH brute-force logic in its default rule set. Suricata could not start with its default 49,828-rule set on a 2 GB VM; the OS killed it. Only after manual rule reduction did it run, after which it caught two of four attacks. Zeek ran cleanly, used barely any resources, and was the only tool to capture the SSH brute force — but it did so silently in a log file rather than raising an alert.

The concurrent resource test produced the clearest single result of the project: running Snort and Zeek at the same time during a SYN flood pushed RAM to 83% and CPU to 78–100%, taking the SSH service offline. On 2 GB hardware, two tools running simultaneously is not a viable configuration.

The right IDS for an SMB isn't the most powerful one — it's the one the administrator can actually keep running.

RECOMMENDATIONS

1 Start with Snort

The only tool that ran reliably on default rules with no expert configuration. Produces real-time alerts for port scans and floods. Compensate for the SSH brute-force blind spot by enforcing SSH key authentication and installing fail2ban for failed-attempt limiting.

2 Add Zeek — but on a separate host

Zeek's forensic depth is genuine and fills the exact gap Snort leaves. Its ssh.log captured every brute-force attempt and identified the attack tool from the client banner. However, the resource test shows that running it alongside Snort on a 2 GB host takes services offline during an attack. It needs its own machine, even if that machine is also small.

3 Avoid Suricata unless hardware and expertise allow

Suricata is fast and capable with enough resources — the literature consistently shows this. But its memory footprint and rule-management complexity put it out of reach for an SMB without a 4 GB+ host or a security specialist available to hand-tune the rule set.

Future work should repeat these experiments across hardware envelopes of 2, 4, and 8 GB RAM to draw a meaningful curve of where each tool becomes practical, and should include encrypted traffic to test performance against modern TLS-wrapped attack patterns.

ACKNOWLEDGEMENTS

All experimental work was conducted inside a self-contained virtual lab with no involvement of external systems, live user data, or real credentials.